

# JavaScript mid-term exam RETAKE (2022-23-1)

**Due** No Due Date      **Points** None

## Important information

- You have **180 minutes** to complete the exam. Filling in the included `README.eng.md` template, reading the tasks, downloading the materials, compressing and uploading your solution is **included** in the time limit.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 19:00 (Hungarian time) and there will be no opportunity to submit your solution after the aforementioned time.**
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)! By filling in the `README.eng.md` file, you accept the possible consequences of cheating.
- The tasks are independent and can be solved **in any order**.
- To begin your work you should first [download the starting package we have prepared](#). Each task has its own directory in the package. All tasks are somewhat prepared. Generally, you only have to edit the provided `.js` files, but feel free to divide up your solution into multiple `.js` files, or even edit the HTML if needed.
- The starting package contains a template called `README.eng.md` where you should fill in your name and Neptun code (the `<>` characters must be removed). **Submissions without a properly filled `README.eng.md` file will not be graded!**
- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

## Task 1: Students (1-students, 8 pts)

The file `index.js` defines the array `students` which contains the following data about each student: their unique education code (`code`), their name (`name`) and the grades they've received on a course (`grades`). Some students may have a different number of grades than others due to their attendance, diligence and other extra work. Solve the following tasks using the array `students`. *Technical help: use of array functions is recommended.*

- a. (2 pts) Output the name of the student whose code is `xrpxmb` to the element with ID `task1`.
- b. (2 pts) Output the names of all students who received no grade other than 5 to the element with ID `task2`.
- c. (2 pts) Output the number of students who failed (received at least three 1s) to the element with ID `task3`.
- d. (2 pts) Output the name of the student who received the highest number of 1s to the element with ID `task4`.

## STUDENTS

Hogy hívják azt a diákot, akinek a kódja "xrpwmb"?

What's the name of the student whose code is "xrpwmb"?

Vittorio Eckhard

Kik azok a diákok, akiknek csupa 5-ösük van?

Who are the students that received no grade other than 5?

Vittorio Eckhard, Philip Roebby

Hány olyan diák van, aki megbukott (legalább 3 darab 1-esé van)?

How many students have failed (received at least three 1s)?

2

Ki kapta a legtöbb egyest?

Who received the highest number of 1s?

Jacobo Fidock

## Task 2: Word game (2-word-game, 8 pts)

There are relatively many five letter words in the English language. These can be used to play various word games. In this game the game master thinks of a five letter word and the others try to guess it by saying five letter words. The game master will respond with the *number of letters that are in the correct position*. Based on this, the others have to narrow down and guess the correct word.

Now you have to create this game in the browser. The file `index.html` already imports the file `words.js` which stores the possible five letter words in the array `wordList`. Solve the following tasks in the `index.js` file.

- a. (1 pt) Choose a random word from the array `wordList` and display it in the div `divTheWord` that is inside the `details` element. (You can use this to view the solution while testing.)
- b. (1 pt) Currently the form on the page is submitted when the user presses ENTER or clicks the button. Prevent this behaviour using JavaScript. Instead of submitting the form, select the text in the text field and delete the content of the `span` element of the form.
- c. (1 pt) When the form is submitted, check that the entered word consists of exactly 5 characters. If not, then display an error message (e.g. "The length of the word is not 5!") in the `span` element of the form.
- d. (1 pt) When the form is submitted, check that the entered word is in the array `wordList`. If not, then display an error message (e.g. "The word is not considered acceptable!") in the `span` element of the form.
- e. (1 pt) Calculate how many letters of the guessed word are in the correct position and output the number to the console.
- f. (1 pt) Insert the guessed word and the calculated number as the first row of the table.
- g. (1 pt) If the guess is correct, apply the style class of `correct` to the row and display the div `end-of-game`.
- h. (1 pt) When the button inside the div `end-of-game` is clicked, reload the page, therefore starting a new game.

## WORD GAME

Guess:   The length of the word is not 5!  
 ▼ Click to see the word  
 horse

WORD MATCHES

---

### Task 3: Romeo and Juliet (3-romeo, 8 pts)

After taking the life of Tybalt, Romeo must flee, but he visits his young wife one last time under the cover of night to say farewell. Everyone in the house is already sleeping, only Juliet is dreaming of her lover by the candlelight. To get her attention, Romeo knocks on her window with a stone so that they can be together one last time. Let's help Romeo hit the illuminated window! To do this, we will create an application using `canvas` where we first determine the strength and direction of the shot, then throw the stone.

- a. (1 pt) Draw the stone (`ball`), bush (`bush`), and windows (`windows`) as rectangles.
- b. (1 pt) Use the appropriate images (`ball`, `bush`) instead of rectangles.
- c. (1 pt) Choose a window (random integer between 0 and 2) and draw it in yellow colour!
- d. (1 pt) As you move the mouse over the canvas, read the mouse's position (`offsetX`, `offsetY`) and draw a 3px wide red line from the `arrow` object's `fx`, `fy` point to the mouse position (you can store this in the `arrow` object's `tx`, `ty` properties).
- e. (1 pt) Start moving the stone when a click occurs. The direction of the red segment will be the direction of the shot and the length of the segment will be the strength of the shot. Give the stone horizontal and vertical velocity and acceleration. The velocities should be given by the x and y extent of the red segment (i.e. the difference between the mouse's x and y coordinates and the arrow object's `fx`, `fy` point). This value should be multiplied by a factor of e.g. 3, to give the stone sufficient speed. The vertical acceleration should be fixed, e.g. 300 px/s<sup>2</sup>.
- f. (1 pt) If the stone touches the bush, stop the movement of the stone (e.g. by setting its speed and acceleration to zero) and write "Oooops!" to the canvas.
- g. (1 pt) If the stone touches a window, also stop the movement. If the touched window is yellow, then write "Come, my lover!" to the canvas, otherwise write "Oooops!".

- h. (1 pt) If the stone touches reaches the ground first, stop its movement and write "Oooops!" to the canvas.

*It can be useful to use a variable to track the game's state. The game can be restarted by reloading the page.*

## ROMEO AND JULIET



## Task 4: Time penalty (4-time-penalty, 8 pts)

In an imaginary competition, participants can receive time penalties for various reasons. We are creating an application that displays and manages these penalties. The file `index.js` contains an example data structure for storing contestants. We store the identification number (`id`), name (`name`), and penalties (`penalties`) for each contestant. The array `penalties` stores the start of each penalty in JavaScript's internal date format (`timestamp`, milliseconds) and the length of the penalty (milliseconds) for each penalty.

*Technical help: JavaScript stores dates as the number of milliseconds since 1970-01-01, and the easiest way to produce this is using the `Date.now()` method. The date that belongs to this internal format can be nicely outputted as follows: `new Date(ms).toLocaleString()`.*

- a. (1 pt) Display the list of contestants in the list `contestants` in the format specified in `index.html`. (`<li data-id="1">Note1 <span>45s</span></li>`)
- b. (1 pt) Display the remaining cumulative penalty time for each contestant at the time the page is loaded. To do this, we must check the difference in seconds between the end of each penalty and the current time, and calculate the sum of the positive values. If the cumulative sum is greater than 0, apply

the `penalty` class to the list item!

- c. (1 pt) When clicking on a contestant, display their details. The clicked contestant's ID can be read from the corresponding data attribute. The details are in the div `contestant-editor`, which is initially hidden. Display it and update the `h2` element's content with the contestant's name. *Technical help: it is advised to store the selected contestant's id in a state variable! This can later be used to index the `contestants` object and find their data.*
- d. (1 pt) Display the selected contestant's list of penalties in the details panel using the format specified in the file `index.html` (start + length, progress bar, remaining time). Set the `max` attribute of the progress element to `duration`, and its `value` to the remaining time (also displayed as text after the progress bar).
- e. (1 pt) Some buttons are already prepared on the details panel. Clicking on them issues another penalty that starts at the current time (`Date.now()`) and its length is determined by the `data-duration` of the button. Update the details panel.
- f. (1 pt) Make it possible to add a new contestant using the "Add new contestant" button.
- g. (1 pt) Refresh the list of contestants and the details panel every second.
- h. (1 pt) Save the `contestants` object to `localStorage` after every change, and read it from `localStorage` if it exists when the page is loaded.

## TIME PENALTY

Alice0s

Bob0s

BOB

- 1/5/2023, 10:41:44 PM + 10s  (0s)
- 1/5/2023, 10:41:46 PM + 10s  (0s)